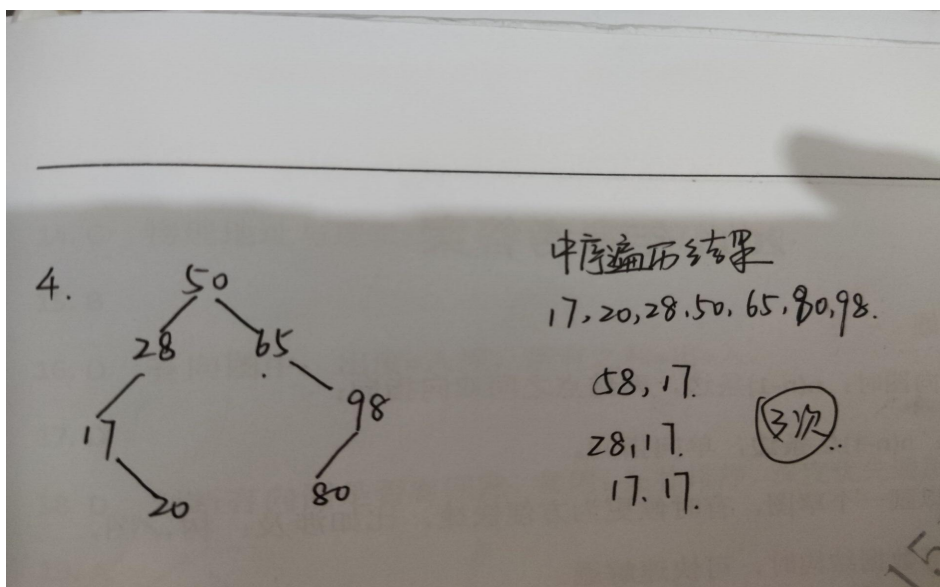
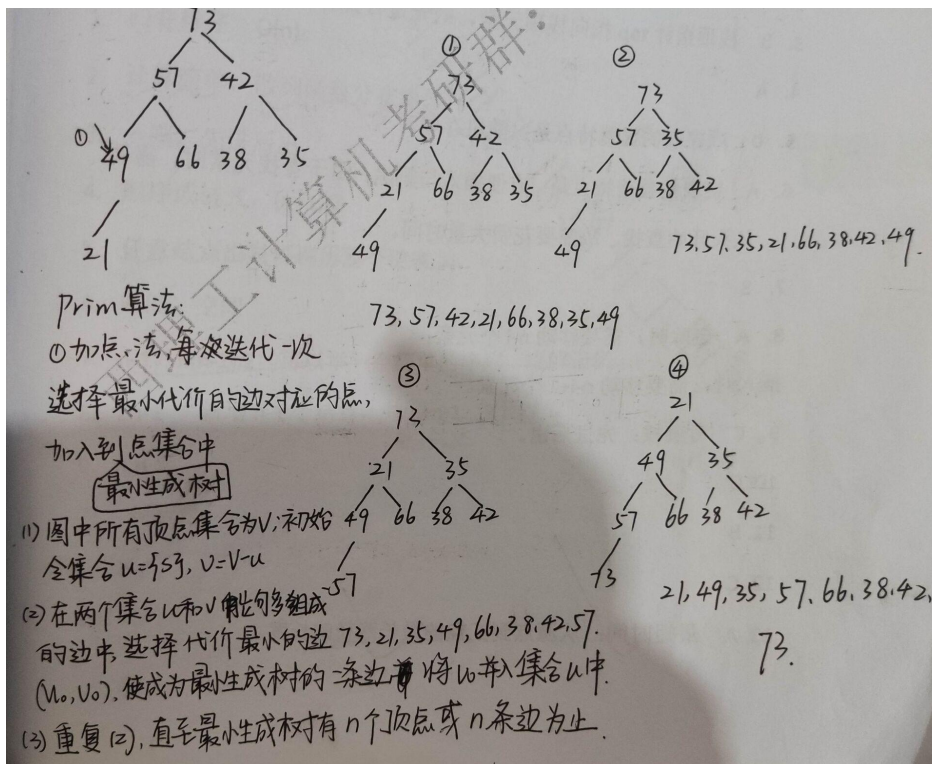




21年

顺序表结构定义

typedef struct

datatype data[MAXSIZE];
int len;

} SeqList;

void Reverse(SeqList &L)

{ datatype temp;

for (int i=0; i<L.len/2; i++)

{ temp = L.data[i];

L.data[i] = L.data[L.len-i-1];

L.data[L.len-i-1] = temp;

}

}

统计值为x的结点个数

typedef struct Lnode {

datatype key;

struct Lnode *next;

} Lnode, *LinkList;

int count(LinkList head, datatype x)

{ int k=0; //统计值为x的个数

LNode *p=head->next;

while (p!=NULL)

{ if (p->key == x)

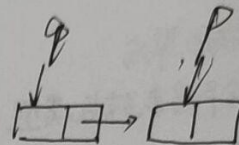
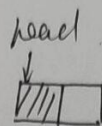
k++;

p=p->next;

return k;

}

顺序表逆置



单链表逆置

void reverse-link(LinkList head)

{ LNode *p=head->next,*q;

head->next=NULL;

while (p!=NULL)

{ q=p;

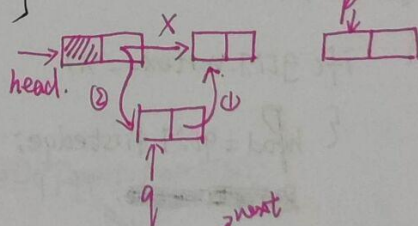
p=p->next;

q->next=head->next; 头插法

head->next=q;

}

}



q=head->next;

head->next=q;

3. 编写算法

// 邻接表结点类型

typedef struct node

{ int adjvex; // 邻接点域

struct node *next; // 指向下一个邻接点的指针域

} EdgeNode;

typedef struct vnode // 顶点表结点类型

{ int vertex; // 顶点域

EdgeNode *firstedge; // 指向邻接表第一个结点的指针域

} VertexNode;

void print (VertexNode g[], int x, int n)

{ EdgeNode ~~head~~, *p;
for (int i = 0; i < n; i++)

{ if (g[i].vertex == x)

{ ~~head~~ = g[i].firstedge;

~~P = head~~; if (P == NULL) { printf("值为x的结点无邻接结点\n");
return; }

while (P != NULL)

{ printf("输出与值为x的结点邻接的顶点的位置: \n");

printf("%d", P->adjvex);

P = P->next;

}

}

}

}